

AutoChip Tutorial Assignment

LLM-assisted Verilog generation with AutoChip trajectories, feedback, and verification

Course: LLM4ChipDesign | Instructors: Ramesh Karri, Weihua Xiao

Overview

In this assignment, you will use the provided **AutoChip** workflow to generate synthesizable Verilog from natural-language specifications and refine the design through multiple generation attempts, tool feedback, and fixes. You are required to complete **all eight tutorial examples** listed below and additionally try the **8-bit CPU example**.

Provided tutorial examples - complete all eight

- Example 1 - binary_to_bcd (combinational converter)
- Example 2 - sequence_detector (FSM sequence matching)
- Example 3 - shift_register (sequential shift register)
- Example 4 - abro_state_machine (ABRO handshake style FSM)
- Example 5 - dice_roller (randomized behavior)
- Example 6 - lfsr (linear feedback shift register)
- Example 7 - sequence_generator (pattern generator)
- Example 8 - traffic_light (traffic light FSM)

Learning goals

- Understand AutoChip's structured multi-step generation trajectory.
- Use compilation and simulation feedback to improve generated RTL across iterations.
- Compare AutoChip behavior across combinational, sequential, FSM, and randomized examples.
- Explore whether the workflow can scale to a more complex 8-bit processor design.

Tools and setup

- **Environment:** the provided AutoChip Colab / notebook workflow.
- **Simulation:** Icarus Verilog (**iverilog** and **vvp**); **gtkwave** is optional.
- **Reproducibility:** the notebook should run top-to-bottom except for adding an API key in the designated place.
- **Security:** do not include API keys or other credentials in your submitted files.

Part I - Complete all eight tutorial examples

Use AutoChip to work through every example below. Keep the example names unchanged in your notebook and report so that the submission is easy to review.

- Example 1 - `binary_to_bcd` (combinational converter)
- Example 2 - `sequence_detector` (FSM sequence matching)
- Example 3 - `shift_register` (sequential shift register)
- Example 4 - `abro_state_machine` (ABRO handshake style FSM)
- Example 5 - `dice_roller` (randomized behavior)
- Example 6 - `lfsr` (linear feedback shift register)
- Example 7 - `sequence_generator` (pattern generator)
- Example 8 - `traffic_light` (traffic light FSM)

For each example, include the following evidence

- **Configuration:** show the relevant AutoChip configuration / parameters used for the run.
- **Specification:** include the natural-language function description and module template given to AutoChip.
- **Testbench:** include the testbench used to evaluate the generated design and explain how it is invoked.
- **Generation trajectory:** preserve the important AutoChip attempts, feedback, and fixes leading to your final result.
- **Verification:** show the exact **iverilog/vvp** command and simulation output whenever the generated design reaches a verifiable state.
- **Short explanation:** describe the most important change AutoChip made during the trajectory.

What if AutoChip does not reach a fully correct design?

A correct final Verilog implementation is preferred, but the **trajectory is also an important part of this assignment**. If AutoChip does not successfully finish an example, preserve the final attempt and clearly explain the remaining problem, the feedback AutoChip received, and what changes were attempted. Do not hide failed attempts that are useful for understanding the generation process.

Part II - Try the 8-bit CPU example

After completing the eight tutorial examples, use AutoChip to try the accumulator-based 8-bit processor design shown below. This task explores how the workflow behaves on a substantially more complex design.

Free chat

- Providing human feedback (the “conversation”) increased success
- Can we leverage this for more complex designs?
- Let’s try make an (8-bit) processor

```
1 Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?
```

Fig. 10. 8-bit accumulator-based processor: Starting co-design prompt

Hammond Pearce

CPU example: accumulator-based 8-bit processor starting co-design prompt.

CPU requirements

- Start from the CPU prompt shown in the figure, or an equivalent prompt that preserves the same constraints.
- Use AutoChip's iterative generation process rather than a single one-shot prompt.
- Preserve the major attempts / trajectory used to develop the processor architecture and RTL.
- Show any compilation, simulation, or tool feedback obtained during the process.
- Summarize the final status: what AutoChip successfully generated, what remained incomplete, and what further refinement would be needed.

Suggested points to discuss

- How AutoChip decomposes or reasons about the ISA, datapath, accumulator, control logic, and I/O constraints.
- Whether feedback that works well for smaller examples remains effective for the CPU.
- Which part of the CPU design is the most difficult for the workflow to complete.

What to submit

Submission method: **GitHub repository only** (no email submission). Please create or use your homework repository and organize this assignment in a folder such as **HW_AutoChip/**.

- **Executed notebook:** a completed notebook file, for example **AutoChip_hw.ipynb**, containing all eight examples and the CPU example.
- **Generated artifacts:** any generated Verilog/code/logs/output files from the notebook runs.
- **Report PDF:** **AutoChip_Report.pdf** summarizing your results, trajectories, and verification.
- **Repo access:** invite **weihuax6@gmail.com** as a collaborator on your repository.

Report requirements (AutoChip_Report.pdf)

- Use the exact names of all eight tutorial examples.
- For each example, summarize the initial specification, important AutoChip parameters, major trajectory steps, and final verification status.
- Highlight at least one example where tool feedback caused a meaningful RTL change across iterations.
- For the CPU example, describe the starting prompt, major trajectory steps, achieved result, and remaining limitations.
- If an example does not reach correct RTL, explain the remaining issue and the attempted fixes.

Grading rubric

Criterion	Points
Completion of all 8 examples + documented AutoChip trajectories	45
8-bit CPU attempt + trajectory and result analysis	25
Verification evidence and use of feedback	15
Clarity, reproducibility, and completeness of submission	15

Important notes

- Do not rename the eight tutorial examples.
- Keep enough intermediate AutoChip output to make the generation trajectory understandable.
- Do not include API keys, tokens, or private credentials.
- Generated RTL should always be treated as potentially incorrect until checked with the provided testbench / simulation flow.